

The Git & GitHub DevOps Reference Manual

A Complete Bible for DevOps Engineers

Version: 1.0
Audience: DevOps engineers from absolute beginner to advanced
Philosophy: Every concept explained simply enough for a five-year-old, then scaled to production-grade engineering.

Table of Contents

Module	Topic	Page
Part 0	Before You Start: Environment Setup	1
Part 1	Core Concepts	2
Part 2	The Essential Daily Workflow	4
Part 3	Branching & Merging	7
Part 4	GitHub Workflow: Push, Pull, PRs	10
Part 5	History, Undoing & Recovery	13
Part 6	Collaboration: Conflicts, Stash & Rebase	16
Part 7	DevOps Branching Patterns	19
Part 8	CI/CD with GitHub Actions	21
Part 9	Release Management: Tags & Releases	24
Part 10	Advanced Topics	26
Part 11	Troubleshooting Guide	28
Part 12	Command Cheat Sheet	30

How to Use This Manual

1. **New to Git?** Start at [Part 0](#) and go in order.
2. **Know the basics?** Jump to [Part 5](#) for undoing, or [Part 6](#) for team collaboration.
3. **Setting up CI/CD?** Go straight to [Part 8](#).
4. **Something broke?** Check [Part 11](#).
5. **Need a quick reminder?** Flip to [Part 12](#).

Every command in this manual shows:

- **Environment:** Where to run it (Terminal / Browser / GitHub UI)
 - **Expected Output:** Exactly what you should see
 - **Common Errors:** What goes wrong and how to fix it
 - **DevOps Context:** Why this matters in production
-
-

Part 0: Before You Start — Environment Setup

Analogy: Before you can drive a car, you need a driver's license, keys, and to know which pedal is which. This section gets you licensed and seated.

0.1 Install Git

Environment: Windows

1. Visit `https://git-scm.com/download/win`
2. Download the installer
3. Run it. Accept all defaults (click **Next** repeatedly)
4. Open **Git Bash** (Start menu → search "Git Bash")

Verify:

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

(Your version number may differ)

Environment: macOS

Open **Terminal** (Cmd+Space, type `terminal`):

```
$ git --version
```

Expected Output:

```
git version 2.39.3
```

If not found, macOS will prompt to install **Command Line Developer Tools**. Click **Install**.

Environment: Linux (Ubuntu / Debian)

```
$ sudo apt update && sudo apt install git -y  
$ git --version
```

Expected Output:

```
git version 2.34.1
```

Environment: Linux (Fedora / CentOS / RHEL)

```
$ sudo dnf install git -y  
$ git --version
```

0.2 Configure Your Identity

Git needs to know who you are so it can label your work.

Environment: Any terminal

```
$ git config --global user.name "Your Full Name"
$ git config --global user.email "your.email@example.com"
```

***Why?** Every commit is signed with your name and email. In a company, this creates an audit trail.*

Verify:

```
$ git config --global user.name
$ git config --global user.email
```

IMPORTANT: Use the **same email** for GitHub.

0.3 Create a GitHub Account

Environment: Web Browser

1. Go to `https://github.com`
2. Click **Sign up**
3. Enter your email, create a password and username
4. Complete email verification

0.4 Set Up SSH Authentication

SSH lets your computer talk to GitHub securely without typing your password every time.

Environment: Terminal

```
$ ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press **Enter** three times (accept default location, no passphrase).

Copy your public key:

macOS:

```
$ pbcopy < ~/.ssh/id_ed25519.pub
```

Windows (Git Bash):

```
$ cat ~/.ssh/id_ed25519.pub | clip
```

Linux:

```
$ cat ~/.ssh/id_ed25519.pub
```

(Select and copy the output)

Add to GitHub:

1. <https://github.com> → Click your profile picture → **Settings**
2. Left sidebar: **SSH and GPG keys**
3. Click **New SSH key**
4. Title: `My Laptop`
5. Key: Paste what you copied
6. Click **Add SSH key**

Test the connection:

```
$ ssh -T git@github.com
```

Type `yes` when asked about authenticity.

Expected Output:

```
Hi yourusername! You've successfully authenticated...
```

0.5 Install a Text Editor

- **Recommended:** VS Code (<https://code.visualstudio.com>)
- **Alternatives:** Notepad++ (Windows), TextEdit (Mac), Nano/Vim (Linux)

0.6 Configure Default Branch Name

Modern Git and GitHub use `main` instead of `master`. Ensure consistency:

```
$ git config --global init.defaultBranch main
```

0.7 Configure Line Endings (Windows Only)

Windows uses different line endings than Linux/Mac. Fix this:

Environment: Windows (Git Bash)

```
$ git config --global core.autocrlf true
```

Environment: macOS / Linux

```
$ git config --global core.autocrlf input
```

Part 1: Core Concepts

1.1 What is Version Control?

5-Year-Old Analogy: *Imagine a magic camera. You build something with blocks, then take a photo. If your baby brother knocks it over, you show the photo to the magic camera and say "Put it back like this!" The camera restores your blocks exactly.*

DevOps Translation: *Version control tracks every change to files, who made it, when, and why. If production breaks, you can restore the exact code from before the break.*

1.2 Key Concepts Dictionary

Simple Term	Technical Term	What It Actually Is
The folder with the camera inside	Repository (Repo)	A directory containing a <code>.git</code> folder that tracks everything
A photo of your work	Commit	A permanent snapshot of your files at one moment in time
The box where you put things before the photo	Staging Area (Index)	A temporary holding area for files you want to include in the next commit
Your messy desk	Working Directory	Your actual files as they exist right now
The photo album	History	All commits, connected like a timeline
A copy of the photo album	Remote	A version of the repo stored on another server (GitHub)
A parallel universe	Branch	An independent line of development
Combining two universes	Merge	Joining one branch's changes into another
Permission slip	Pull Request (PR)	Requesting to merge your branch into main, with code review

1.3 The Three States of Git

Every file in Git is in one of three states:

Working Directory (modified)	--git add--> stage	Staging Area (staged)	--git commit--> commit	Git Repository (History) (committed)
---------------------------------	-----------------------	--------------------------	---------------------------	---

1. **Working Directory:** You edit files here. Git notices changes but doesn't save them yet.
2. **Staging Area:** You choose which changes to include in your next snapshot.
3. **Repository:** Committed snapshots live here forever.

Restaurant Analogy:

- Working Directory = The prep table where you're chopping vegetables

- *Staging Area = The plate the chef arranged and is about to serve*
- *Commit = The photo that goes on the menu (permanent record)*

1.4 The Lifecycle of a File

Untracked (new)	--git add-->	Staged (ready)	--git commit-->	Unmodified (saved)	--edit-->	Modified (updated)
--------------------	--------------	-------------------	-----------------	-----------------------	-----------	-----------------------

1.5 Understanding `.git`

When you run `git init`, Git creates a hidden folder called `.git`.

This is Git's brain. It stores:

- *Every commit ever made*
- *Every branch*
- *Configuration*
- *The staging area*

NEVER delete `.git` unless you want to erase all history.

To see it:

```
$ ls -la
```

Expected Output:

```
drwxr-xr-x  .git
-rw-r--r--  yourfile.txt
```

Part 2: The Essential Daily Workflow

These are the commands you will use every single day as a DevOps engineer.

2.1 Creating a New Repository

Environment: Terminal

```
$ mkdir my-project  
$ cd my-project  
$ git init
```

Expected Output:

```
Initialized empty Git repository in /home/you/my-project/.git/
```

DevOps Context: You run `git init` when starting a new project, creating a new deployment script repo, or setting up infrastructure-as-code.

2.2 Checking Your Status

This is the single most important command. Run it constantly.

```
$ git status
```

Expected Output (clean repo):

```
On branch main  
nothing to commit, working tree clean
```

Expected Output (untracked file):

```
On branch main  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    new-file.txt
```

Expected Output (modified file):

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
   modified:   existing-file.txt
```

Expected Output (staged file):

```
On branch main
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
   new file:   new-file.txt
```

2.3 Adding Files to the Staging Area

Add one file:

```
$ git add filename.txt
```

Add all files in current directory:

```
$ git add .
```

Add all modified files:

```
$ git add -A
```

DevOps Context: Stage files intentionally. Don't blindly `git add .` — review what you're committing first.

2.4 Unstaging Files

Oops, you staged the wrong file:

```
$ git restore --staged filename.txt
```

Before (staged):

```
Changes to be committed:
  new file:   filename.txt
```

After:

```
Untracked files:
  filename.txt
```

2.5 Committing Your Work

```
$ git commit -m "Describe what you changed"
```

Example — Good messages:

```
$ git commit -m "Add nginx configuration for production"
$ git commit -m "Fix database connection timeout in staging"
$ git commit -m "Update Terraform module to v3.2.1"
```

Example — Bad messages:

```
$ git commit -m "asdf"           # Meaningless
$ git commit -m "stuff"          # What stuff?
$ git commit -m "fixed bug"      # Which bug?
$ git commit -m "update"         # Update what?
```

Multi-line commit (for detailed changes):

```
$ git commit -m "Subject line under 50 chars" -m "Body: detailed explanation of what"
```

DevOps Context: In incident response, clear commit messages save hours. "Fix load balancer health check" is infinitely more useful than "fix stuff."

2.6 Viewing History

Full log:

```
$ git log
```

Compact one-line format (most useful):

```
$ git log --oneline
```

With graph (shows branch structure):

```
$ git log --oneline --graph --all
```

Last 5 commits:

```
$ git log --oneline -5
```

Show what changed in each commit:

```
$ git log --oneline --stat
```

Show file-by-file patch:

```
$ git log --oneline --patch
```

Search commits by message:

```
$ git log --oneline --grep="nginx"
```

View commits by a specific author:

```
$ git log --oneline --author="Your Name"
```

2.7 Seeing What Changed (Before Committing)

Show unstaged changes:

```
$ git diff
```

Show staged changes (what will be committed):

```
$ git diff --staged
```

Show changes in a specific file:

```
$ git diff filename.txt
```

2.8 Ignoring Files (`.gitignore`)

Create a `.gitignore` file to tell Git "never track these files."

Environment: Terminal

```
$ cat > .gitignore << 'EOF'
# Operating system files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Environment variables (NEVER commit secrets)
.env
.env.local

# Dependencies
node_modules/
vendor/

# Build outputs
dist/
build/
*.exe

# IDE files
.vscode/
.idea/
*.swp

# Terraform
.terraform/
*.tfstate
*.tfstate.*
EOF

$ git add .gitignore
$ git commit -m "Add .gitignore for common files"
```

CRITICAL DevOps Rule: Never commit passwords, API keys, `.env` files, or Terraform state files. Use `.gitignore` to protect yourself.

2.9 Removing Files

Remove from Git AND delete from disk:

```
$ git rm old-file.txt
$ git commit -m "Remove deprecated configuration"
```

Remove from Git but KEEP on disk:

```
$ git rm --cached secret-file.txt
$ git commit -m "Remove accidentally committed secret"
```

Then add it to `.gitignore` immediately.

Part 3: Branching & Merging

3.1 What is a Branch?

5-Year-Old Analogy: You're reading a choose-your-own-adventure book. The main story is one path. You put a bookmark at page 50, try a different adventure, and if it's good, you rewrite the main story to include it. The bookmarked path = a branch.

DevOps Translation: Branches let you experiment safely. You NEVER edit production (`main`) directly. You create a branch, test your changes, then merge.

3.2 Basic Branch Commands

List branches:

```
$ git branch
```

Expected Output:

```
feature-auth
hotfix-timeout
* main
```

* = You are currently on this branch.

Create a new branch:

```
$ git branch feature-new-thing
```

Switch to a branch:

```
$ git switch feature-new-thing
```

Create AND switch (shortcut):

```
$ git switch -c feature-new-thing
```

(The `-c` flag means "create")

Delete a merged branch:

```
$ git branch -d feature-new-thing
```

Delete an unmerged branch (force):

```
$ git branch -D feature-new-thing
```

Rename current branch:

```
$ git branch -m better-name
```

3.3 Understanding HEAD

HEAD is a pointer to your current commit.

- HEAD = "You are here"
- HEAD~1 = "One commit before where you are"
- HEAD~3 = "Three commits before where you are"

3.4 Merging Branches

Fast-forward merge (main hasn't changed since you branched):


```
$ git switch main
$ git merge feature-new-thing
```

Expected Output:

```
Updating abc123..def456
Fast-forward
 newfile.txt | 10 ++++++++
 1 file changed, 10 insertions(+)
```

DevOps Context: *Fast-forward is clean and simple. No merge commit needed.*

Three-way merge (main has new commits):

```
$ git switch main
$ git merge feature-new-thing
```

Expected Output:

```
Merge made by the 'ort' strategy.
 newfile.txt | 10 ++++++++
 1 file changed, 10 insertions(+)
```

This creates a **merge commit** — a new commit that has two parents.

Avoid merge commits with rebase (see Part 6).

3.5 Branch Naming Conventions

Pattern	Example	When to Use
feature/description	feature/user-auth	New features
bugfix/description	bugfix/login-timeout	Bug fixes
hotfix/description	hotfix/critical-db-fix	Production emergencies
release/version	release/v2.1.0	Release preparation
docs/description	docs/api-examples	Documentation updates

3.6 Comparing Branches

See what `feature` has that `main` doesn't:

```
$ git log main..feature --oneline
```

See differences between branches:

```
$ git diff main..feature
```

3.7 The Complete Branch Workflow

```
# 1. Start from latest main
$ git switch main
$ git pull

# 2. Create feature branch
$ git switch -c feature/my-change

# 3. Do your work
# ... edit files ...

# 4. Commit
$ git add .
$ git commit -m "Add my feature"

# 5. Push branch
$ git push -u origin feature/my-change

# 6. (On GitHub) Open Pull Request → Review → Merge

# 7. Pull latest main
$ git switch main
$ git pull

# 8. Delete local branch
$ git branch -d feature/my-change
```

Part 4: GitHub Workflow — Push, Pull, PRs

4.1 Connecting to a Remote

Add a remote (typically done once per repo):

```
$ git remote add origin git@github.com:USERNAME/REPO.git
```

Verify remotes:

```
$ git remote -v
```

Expected Output:

```
origin git@github.com:USERNAME/REPO.git (fetch)
origin git@github.com:USERNAME/REPO.git (push)
```

Change remote URL:

```
$ git remote set-url origin git@github.com:NEW-USER/NEW-REPO.git
```

4.2 Pushing Code

Push current branch to remote:

```
$ git push
```

Push a new branch for the first time:

```
$ git push -u origin feature-name
```

The `-u` flag (short for `--set-upstream`) links your local branch to the remote branch. After this, just `git push` works.

Force push (DANGEROUS — rewrites history):

```
$ git push --force
```

DevOps Warning: Only force push on personal feature branches. Never force push to `main` or shared branches.

Force push with lease (safer):

```
$ git push --force-with-lease
```

4.3 Pulling Code

Pull (fetch + merge):

```
$ git pull
```

Fetch only (download without merging):

```
$ git fetch
```

Fetch a specific branch:

```
$ git fetch origin feature-name
```

Pull with rebase (cleaner history):

```
$ git pull --rebase
```

4.4 Cloning a Repository

Clone a remote repo to your local machine:

```
$ git clone git@github.com:USERNAME/REPO.git
```

Clone into a specific folder:

```
$ git clone git@github.com:USERNAME/REPO.git custom-folder-name
```

Clone only the latest commit (faster for CI):

```
$ git clone --depth 1 git@github.com:USERNAME/REPO.git
```

4.5 The Pull Request (PR) Workflow

The PR workflow is the backbone of professional DevOps. No one pushes directly to `main`.

Step-by-Step:

1. Push your branch:

```
bash $ git push -u origin feature-name
```

2. Open PR on GitHub:

- Go to repo on <https://github.com>
- Click "**Compare & pull request**"
- Add title and description
- Click **Create pull request**

3. Code review happens:

- Team members review the **Files changed** tab
- They leave comments on specific lines
- You make fixes, commit, push (PR updates automatically)

4. CI runs (GitHub Actions):

- Tests must pass
- Linting must pass

5. Approve and merge:

- Reviewer clicks **Approve**
- Click **Merge pull request**
- Choose merge strategy:
 - **Create a merge commit** (preserves branch history)
 - **Squash and merge** (combines all commits into one)
 - **Rebase and merge** (replays commits on main)

6. Clean up:

- Delete the remote branch
- Pull locally:

```
bash $ git switch main $ git pull $ git branch -d feature-name
```

4.6 Forking vs. Cloning

	Clone	Fork
What	Copy to your laptop	Copy on GitHub to your account
When	Working on your team's repo	Contributing to open source
Example	<pre>git clone git@github.com:mycompany/ project.git</pre>	Click "Fork" on <code>github.com/ kubernetes/kubernetes</code>

After forking, clone your fork:

```
$ git clone git@github.com:YOUR-NAME/original-repo.git
```

4.7 Syncing a Fork

```
# Add the original repo as "upstream"  
$ git remote add upstream git@github.com:original-owner/original-repo.git  
  
# Fetch latest from original  
$ git fetch upstream  
  
# Merge into your main  
$ git switch main  
$ git merge upstream/main  
  
# Push to your fork  
$ git push
```

Part 5: History, Undoing & Recovery

5.1 Viewing History in Detail

Show commit with changes:

```
$ git show COMMIT_ID
```

Show only files changed:

```
$ git show COMMIT_ID --stat
```

Show who changed each line of a file:

```
$ git blame filename.txt
```

Expected Output:

```
abc1234 (Your Name  2024-01-15 09:00:00 +0000  1) Line one content  
def5678 (Teammate   2024-01-16 14:30:00 +0000  2) Line two content
```

DevOps Context: *git blame* is essential during incidents. "Who changed this config line 3 days ago?"

Search all commits for a string:

```
$ git log --all --grep="timeout"  
$ git log --all -S "database_url" # Search code content, not just messages
```

5.2 Reverting a Commit (Safe Undo)

Rule: *Revert is always safe. It creates a NEW commit that undoes an OLD commit. History is preserved.*

Revert the most recent commit:

```
$ git revert HEAD
```

Revert a specific commit:


```
$ git revert abc1234
```

An editor opens for the commit message. Save and exit.

Revert without editing commit message:

```
$ git revert HEAD --no-edit
```

Revert a merge commit:

```
$ git revert -m 1 MERGE_COMMIT_ID
```

DevOps Context: *Revert is your go-to for production incidents. It undoes a bad deploy instantly while keeping the audit trail.*

5.3 Resetting (Use With Caution)

Reset moves the branch pointer. It can erase commits.

Soft reset (keep changes staged):

```
$ git reset --soft HEAD~1
```

Mixed reset (keep changes unstaged):

```
$ git reset --mixed HEAD~1
```

Hard reset (DESTROY changes — DANGEROUS):

```
$ git reset --hard HEAD~1
```

DevOps Warning: `git reset --hard` permanently deletes uncommitted work. Never use on shared history.

5.4 The Reflog — Your Safety Net

Analogy: The reflog is like a backup camera that recorded everything, even things you thought you deleted.

View all actions (even after reset):

```
$ git reflog
```

Expected Output:

```
abc1234 HEAD@{0}: commit: Add feature
def5678 HEAD@{1}: commit: Fix bug
1234567 HEAD@{2}: reset: moving to HEAD~1
```

Recover a lost commit:

```
$ git reset --hard HEAD@{2}
```

View reflog for a specific branch:

```
$ git reflog show main
```

DevOps Context: `git reflog` has saved many engineers. If you accidentally reset `main`, the reflog remembers.

5.5 Cherry-Picking

Apply a specific commit from one branch to another.

```
# On main, apply a commit from feature
$ git switch main
$ git cherry-pick abc1234
```

Cherry-pick without committing (just apply):

```
$ git cherry-pick -n abc1234
```

DevOps Context: Cherry-pick a hotfix from `main` to a `release` branch without merging everything.

5.6 Amending the Last Commit

Oops, typo in the commit message:

```
$ git commit --amend -m "Correct message"
```

Forgot to add a file:

```
$ git add forgotten-file.txt  
$ git commit --amend --no-edit
```

DevOps Context: Only amend unpushed commits. Amending pushed commits rewrites public history.

Part 6: Collaboration — Conflicts, Stash & Rebase

6.1 Merge Conflicts

What Causes a Conflict?

When two branches change the **same line** of the **same file**, Git doesn't know which version to keep.

Example conflict markers:

```
<<<<<<< HEAD
This is what YOUR branch has
=====
This is what THE OTHER branch has
>>>>>>> other-branch-name
```

How to Resolve

1. Find conflicts:

```
bash $ git status
```

Look for: "Unmerged paths"

2. Open the file and edit it. Remove the `<<<<<<<`, `=====`, and `>>>>>>>` markers.

3. Stage the resolved file:

```
bash $ git add filename.txt
```

4. Complete the merge:

```
bash $ git commit -m "Resolve merge conflict in filename.txt"
```

Search for conflict markers:

```
$ grep -r "<<<<<<<" .
```

Abort a merge in progress:

```
$ git merge --abort
```

DevOps Context: Conflicts are normal. Communicate with teammates about who owns which files.

6.2 Git Stash

Temporarily save work without committing.

Stash current changes:

```
$ git stash
```

Expected Output:

```
Saved working directory and index state ...
```

Stash with a message:

```
$ git stash push -m "WIP: updating terraform config"
```

List stashes:

```
$ git stash list
```

Expected Output:

```
stash@{0}: WIP on main: abc1234 Add feature  
stash@{1}: On main: WIP terraform migration
```

Apply and remove latest stash:

```
$ git stash pop
```

Apply but keep stash:

```
$ git stash apply
```

Apply specific stash:

```
$ git stash apply stash@{1}
```

Drop a stash:

```
$ git stash drop stash@{0}
```

Clear all stashes:

```
$ git stash clear
```

DevOps Context: *Stash when someone says "Production is down, fix it NOW" but you're in the middle of a refactor.*

6.3 Rebasing

Rebase replays your commits on top of another branch. It creates a cleaner, linear history.

Basic rebase:

```
# On your feature branch
$ git switch feature-name
$ git rebase main
```

Interactive rebase (rewrite history):

```
# Rebase the last 3 commits
$ git rebase -i HEAD~3
```

An editor opens. You can:

- `pick` — keep commit (default)
- `reword` — edit commit message
- `squash` — combine with previous commit
- `drop` — remove commit
- `fixup` — combine and discard message

Continue rebase after resolving conflicts:

```
$ git rebase --continue
```

Abort rebase:

```
$ git rebase --abort
```

DevOps Context: *Rebase feature branches before opening a PR. This eliminates unnecessary merge commits. **NEVER rebase shared branches like** `main`.*

6.4 Interactive Rebase Example

Clean up messy commits before a PR:

```
$ git rebase -i HEAD~4
```

Editor shows:

```
pick abc1234 Fix typo
pick def5678 Add config
pick ghi901 Add tests
pick jkl234 Update docs
```

Edit to:

```
reword abc1234 Fix typo
squash def5678 Add config
pick ghi901 Add tests
fixup jkl234 Update docs
```

Result: Clean commits with good messages.

Part 7: DevOps Branching Patterns

7.1 Feature Branch Workflow

Best for: Small teams, startups, most modern DevOps.

```
main (always deployable)
├── feature/login
├── feature/dashboard
└── bugfix/memory-leak
```

Rules:

- `main` is sacred — always works
- All work happens on feature branches
- Every merge goes through PR + code review

7.2 GitFlow

Best for: Large teams with scheduled releases.

```
main          (production)
develop       (integration)
├── feature/...
└── feature/...
hotfix/...    (emergency fixes for production)
release/...   (release preparation)
```

Rules:

- `main` = production only
- `develop` = integration branch
- Features branch from `develop`
- Releases branch from `develop`, merge to `main`
- Hotfixes branch from `main`, merge back to both

7.3 Trunk-Based Development

Best for: CI/CD-heavy teams deploying multiple times daily.

```
main (trunk)
├── short-lived feature branches (hours, not days)
```

Rules:

- Everyone commits to `main` or very short-lived branches
- Feature flags hide incomplete work
- Requires robust CI

7.4 Forking Workflow

Best for: Open source projects.

1. Fork the official repo to your GitHub account
 2. Clone YOUR fork locally
 3. Push branches to YOUR fork
 4. Open PR from your fork → official repo
-
-

Part 8: CI/CD with GitHub Actions

8.1 What is GitHub Actions?

GitHub Actions is a built-in CI/CD platform. It runs code (workflows) in response to Git events (push, PR, schedule).

Analogy: A robot watches your mailbox. Every time you put a letter in, the robot opens it, checks the spelling, rewrites it if needed, and approves or rejects it.

8.2 Workflow Structure

Workflow files live in `.github/workflows/` and use YAML.

```

name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: make test

```

Key Components:

Component	Description
name	Display name in GitHub UI
on	Events that trigger the workflow
jobs	Groups of steps
runs-on	Type of virtual machine
steps	Individual commands
uses	Reusable action from GitHub Marketplace
run	Shell command

8.3 Common Triggers

```

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 0 * * 0' # Every Sunday at midnight
  workflow_dispatch: # Manual trigger button

```

8.4 Complete DevOps Workflow Example

```
name: Deploy to Production

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to server
        run: |
          echo "Deploying to production..."
          # Your deploy scripts here
```

8.5 Branch Protection with Required Status Checks

1. GitHub repo → **Settings** → **Branches**
2. Add rule for `main`
3. Enable:
 - Require pull request before merging

- Require status checks to pass
- Select your workflow name (e.g., `CI Pipeline`)
- Require code review

8.6 Secrets and Environment Variables

Store sensitive data in GitHub settings, NOT in code.

1. Repo → **Settings** → **Secrets and variables** → **Actions**
2. Click **New repository secret**
3. Name: `PROD_API_KEY`
4. Value: Your actual secret

Access in workflow:

```
- name: Deploy
  run: |
    curl -H "Authorization: Bearer ${ secrets.PROD_API_KEY }" ...
```

8.7 Caching for Speed

Cache dependencies to speed up workflows:

```
- name: Cache node modules
  uses: actions/cache@v3
  with:
    path: ~/.npm
    key: ${ runner.os }-node-${ hashFiles('**/package-lock.json') }
```

Part 9: Release Management — Tags & Releases

9.1 Tags

Tags mark specific commits as releases.

Lightweight tag (just a name):

```
$ git tag v1.0.0
```

Annotated tag (with message):

```
$ git tag -a v1.0.0 -m "Release version 1.0.0"
```

List tags:

```
$ git tag
```

Push a single tag:

```
$ git push origin v1.0.0
```

Push all tags:

```
$ git push origin --tags
```

Delete a local tag:

```
$ git tag -d v1.0.0
```

Delete a remote tag:

```
$ git push --delete origin v1.0.0
```

9.2 Semantic Versioning

Format: MAJOR.MINOR.PATCH

Version	Meaning
1.0.0	First stable release
1.1.0	New features, backward compatible
1.1.1	Bug fixes only
2.0.0	Breaking changes

9.3 Creating a GitHub Release

1. Go to repo on GitHub → **Releases** (right sidebar)
2. Click **Draft a new release**
3. Choose a tag (or create new)
4. Add release title and notes
5. Attach binaries (optional)
6. Click **Publish release**

9.4 Release Branch Strategy

```
# Create release branch
$ git switch -c release/v2.1.0

# Fix any last-minute issues
$ git commit -m "Bump version to 2.1.0"

# Merge to main and tag
$ git switch main
$ git merge release/v2.1.0
$ git tag -a v2.1.0 -m "Release 2.1.0"
$ git push origin main --tags

# Merge back to develop
$ git switch develop
$ git merge release/v2.1.0
```

Part 10: Advanced Topics

10.1 Git Hooks

Scripts that run automatically on Git events.

Available hooks:

- `pre-commit` — Run before each commit
- `prepare-commit-msg` — Modify commit message

- `post-merge` — Run after merge
- `pre-push` — Run before push

Example: Pre-commit hook to run linter

```
# Create the hook
$ cat > .git/hooks/pre-commit << 'EOF'
#!/bin/bash
echo "Running linter..."
make lint || exit 1
EOF

$ chmod +x .git/hooks/pre-commit
```

Note: Hooks in `.git/hooks` are not shared. Use [Husky](#) or [pre-commit](#) for team-wide hooks.

10.2 Submodules

Include another repo inside your repo.

Add a submodule:

```
$ git submodule add git@github.com:user/shared-scripts.git scripts
```

Clone with submodules:

```
$ git clone --recurse-submodules git@github.com:user/repo.git
```

Update submodules:

```
$ git submodule update --remote
```

10.3 Worktrees

Work on multiple branches simultaneously without cloning multiple times.

```
# Create a new directory for feature branch work
$ git worktree add ../my-project-feature feature-branch

# Now you have two directories, each on a different branch
$ cd ../my-project-feature
$ git branch
# Shows: * feature-branch
```

List worktrees:

```
$ git worktree list
```

Remove a worktree:

```
$ git worktree remove ../my-project-feature
```

10.4 Bisect (Find the Bug)

Binary search through history to find which commit introduced a bug.

```
# Start bisect
$ git bisect start

# Mark current (bad) commit
$ git bisect bad

# Mark last known good commit
$ git bisect good v1.0.0

# Git checks out a middle commit
# Test it, then mark good or bad:
$ git bisect good    # OR git bisect bad

# Repeat until Git identifies the first bad commit

# Reset when done
$ git bisect reset
```


Part 11: Troubleshooting Guide

"fatal: not a git repository"

Cause: You ran a Git command outside a repository.

Fix:

```
$ cd /path/to/your/repo
# OR
$ git init # If you haven't initialized yet
```

"Permission denied (publickey)"

Cause: SSH key not set up or not added to GitHub.

Fix:

1. Generate SSH key: `ssh-keygen -t ed25519 -C "email"`
2. Add to GitHub: Settings → SSH keys
3. Test: `ssh -T git@github.com`

"Merge conflict"

Fix:

```
# Find conflicted files
$ git status

# Open each file, remove <<<<<< ===== >>>>>> markers
# Edit to desired result

# Stage resolved files
$ git add .

# Complete merge
$ git commit -m "Resolve merge conflicts"
```

"Your branch is ahead of 'origin/main' by 3 commits"

Fix:

```
$ git push
```

"Your branch and 'origin/main' have diverged"

Fix:

```
# Option 1: Merge (preserves branch structure)
$ git pull

# Option 2: Rebase (cleaner history)
$ git pull --rebase
```

"The requested URL returned error: 403"

Cause: Wrong permissions or using HTTPS without token.

Fix: Switch to SSH:

```
$ git remote set-url origin git@github.com:USER/REPO.git
```

"Failed to push some refs"

Cause: Remote has commits you don't have locally.

Fix:

```
$ git pull
# Resolve any conflicts
$ git push
```

"I committed to the wrong branch"

Fix:

```
# Create the correct branch (includes the commit)
$ git switch -c correct-branch

# Go back to original branch and remove commit
$ git switch wrong-branch
$ git reset --soft HEAD~1
```

"I committed a secret/password"

Fix:

```
# If NOT pushed yet
$ git reset --soft HEAD~1

# If ALREADY pushed (complex – requires history rewrite)
# Use GitHub's secret scanning or BFG Repo-Cleaner
# Rotate the secret immediately regardless
```

ALSO:

```
$ git rm --cached secret-file
$ git commit -m "Remove secret"
$ echo "secret-file" >> .gitignore
$ git add .gitignore
$ git commit -m "Add secret to .gitignore"
```

"git log shows a mess of merge commits"

Fix: Use rebase before merging:

```
$ git switch feature
$ git rebase main
$ git switch main
$ git merge feature
```

Workflow fails with "Permission denied"

Cause: GitHub Actions token lacks permissions.

Fix: Add permissions to workflow:

```
permissions:
  contents: write
  pull-requests: read
```

Part 12: Command Cheat Sheet

Daily Commands

Command	Description
<code>git status</code>	Check what's changed
<code>git add filename</code>	Stage a file
<code>git add .</code>	Stage all changes
<code>git commit -m "msg"</code>	Commit staged changes
<code>git push</code>	Upload commits
<code>git pull</code>	Download and merge latest
<code>git log --oneline</code>	View compact history
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes

Branch Commands

Command	Description
<code>git branch</code>	List branches
<code>git branch name</code>	Create branch
<code>git switch name</code>	Switch branch
<code>git switch -c name</code>	Create and switch
<code>git merge name</code>	Merge branch into current
<code>git branch -d name</code>	Delete merged branch
<code>git branch -D name</code>	Force delete branch

History & Undo

Command	Description
<code>git log --oneline</code>	Compact history
<code>git show COMMIT</code>	Show commit details
<code>git blame FILE</code>	Who edited each line
<code>git revert COMMIT</code>	Undo commit safely
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes
<code>git reset --hard HEAD~1</code>	Delete last commit AND changes
<code>git reflog</code>	View all actions
<code>git cherry-pick COMMIT</code>	Apply specific commit
<code>git commit --amend</code>	Edit last commit

Stashing

Command	Description
<code>git stash</code>	Save current work
<code>git stash push -m "note"</code>	Save with message
<code>git stash list</code>	List stashes
<code>git stash pop</code>	Apply and remove stash
<code>git stash apply</code>	Apply but keep stash
<code>git stash drop</code>	Delete stash
<code>git stash clear</code>	Delete all stashes

Rebasing

Command	Description
<code>git rebase main</code>	Replay commits on main
<code>git rebase -i HEAD~3</code>	Interactive rebase
<code>git rebase --continue</code>	Continue after conflict
<code>git rebase --abort</code>	Cancel rebase

Remote

Command	Description
<code>git remote -v</code>	Show remotes
<code>git remote add NAME URL</code>	Add remote
<code>git fetch</code>	Download from remote
<code>git pull</code>	Fetch + merge
<code>git push -u origin BRANCH</code>	Push and set upstream
<code>git clone URL</code>	Copy repository

Tags & Releases

Command	Description
<code>git tag</code>	List tags
<code>git tag v1.0.0</code>	Create lightweight tag
<code>git tag -a v1.0.0 -m "msg"</code>	Create annotated tag
<code>git push origin v1.0.0</code>	Push tag
<code>git push origin --tags</code>	Push all tags

Glossary: From 5-Year-Old to Engineer

Kid-Friendly	Professional	Real Definition
Magic Camera	Version Control	System that saves and restores file history
Photo	Commit	A permanent snapshot of files
Prep Box	Staging Area	Where you prepare files for a commit
Photo Album	Repository	All commits, branches, and history
School Library	Remote (GitHub)	Server copy of your repo
Bookmarked Path	Branch	An independent line of development
Combine Paths	Merge	Join two branches together
Permission Slip	Pull Request	Formal request to merge into main
Robot	CI/CD	Automated testing and deployment
Recipe	YAML Workflow	Instructions for the robot

This manual is a living document. Git changes. Best practices evolve. Keep learning, commit often, and may your deploys always be green.